

Laborator 3

În Databricks datele sunt organizate pe cataloage și scheme. Dataset-urile demo sunt în catalogul samples.

--HDFS

SHOW DATABASES;

USE samples;

SHOW TABLES;

DESCRIBE TABLE nyctaxi.trips;

SHOW TABLES IN samples.tpcds_sf1000;

SELECT COUNT(*)

FROM samples.tpcds_sf1000.store_sales;

SELECT SUM(ss_quantity)

FROM samples.tpcds_sf1000.store_sales;

```
20 | SHOW TABLES IN samples.tpcds_sf1000;
21
22 | SELECT COUNT(*)
23 | FROM samples.tpcds_sf1000.store_sales;
24
25
26 | SELECT SUM(ss_quantity)
27 | FROM samples.tpcds_sf1000.store_sales;
```

Add parameter

Results 3 of 3		Table	+
	sum(ss_quantity)	Download CSV	>
1	13889504411	Download Excel	>
1 row 6.35s runtime		Copy results to clipboard	

Acest meniu permite exportul rezultatului final al interogării SQL. Deși interogarea a procesat volume mari de date în mod distribuit, doar rezultatul agregat este descărcat local, nu întregul dataset Big Data.

Interogarea `SELECT SUM(ss_quantity) FROM samples.tpcds_sf1000.store_sales` reprezintă un exemplu clasic de procesare Big Data de tip **batch**, care declanșează execuția unui **job Spark distribuit** asupra unui volum foarte mare de date. Tabelul `store_sales` face parte din benchmark-ul TPC-DS la scară SF1000 și conține aproximativ **2,88 miliarde de înregistrări**, fiind stocat într-un format columnar optimizat (Parquet/Delta) și împărțit în peste **1.800 de fișiere** distribuite.

La rularea acestei interogări, Spark nu se poate baza exclusiv pe metadate, așa cum se întâmplă în unele cazuri pentru `COUNT(*)`, deoarece operația `SUM(ss_quantity)` necesită citirea efectivă a valorii coloanei `ss_quantity` din **fiecare rând al tabelului**. Din acest motiv, motorul Spark este obligat să realizeze un **full table scan**, citind toate partițiile tabelului. Procesul de execuție începe cu analiza interogării de către driver-ul Spark, care construiește planul logic și fizic de execuție. Tabelul este împărțit în mai multe partiții, iar pentru fiecare partiție Spark creează unul sau mai multe **task-uri**. Aceste task-uri sunt distribuite către executors (nodurile de lucru din cluster), unde rulează **în paralel**. Fiecare task citește datele din fișierul sau partiția sa și calculează o sumă parțială a valorilor `ss_quantity`.

După finalizarea procesării locale, rezultatele parțiale sunt trimise înapoi către driver, unde are loc etapa de **agregare finală** (reduce), prin însumarea tuturor valorilor parțiale. Rezultatul final este o singură valoare numerică, care reprezintă suma totală a cantităților vândute din întregul set de date.

Indicatorii din profilul de execuție confirmă acest comportament: aproximativ **2,88 miliarde de rânduri citite**, **2,86 GB de date procesate** și **1.837 fișiere parcurse**, toate acestea într-un timp de aproximativ **10–11 secunde**. Acest timp relativ redus pentru un volum atât de mare de date evidențiază avantajele procesării distribuite și ale formatelor Big Data optimizate, precum și eficiența execuției în paralel oferite de Spark.

Din punct de vedere conceptual, această interogare urmează modelul clasic **MapReduce**: etapa de tip *Map* constă în scanarea partițiilor și calculul sumelor locale, iar etapa de tip *Reduce* constă în agregarea rezultatelor parțiale într-un rezultat final. Diferența față de Hadoop MapReduce clasic constă în faptul că Spark minimizează operațiile pe disc și utilizează procesarea în memorie, ceea ce conduce la performanțe mult superioare.

Prin acest exemplu se demonstrează clar capacitatea platformelor Big Data moderne de a procesa volume masive de date într-un timp rezonabil, precum și modul în care o interogare aparent simplă poate declanșa un mecanism complex de execuție distribuită la nivel de cluster.

✓ Finished Cache

mironela.pirnaeu@prof.utm.ro · UTM Shared SQL Warehouse - 2XS · Run on DBSQL

ID: 01f0e7f8-9967-1c3c-8472-8926ca6b23f9



```
SELECT
  SUM(ss_quantity)
...2 more lines
```



Query profile is unavailable for cached results.

You can view the original query using the [Statement ID](#)

Copy ID

Query wall-clock duration ⓘ

Total wall-clock duration 9 s 273 ms

> Scheduling ⓘ 53% 4 s 960 ms

Optimizing query & pruning files ⓘ 43% 3 s 962 ms

See query profile

```
SELECT SUM(ss_quantity)
FROM samples.tpcds_sf1000.store_sales
WHERE ss_quantity > 0;
```

Finished

mironela.pirnaeu@prof.utm.ro · UTM Shared SQL Warehouse - 2XS · Run on DBSQL

Bytes read 2.86 GB	Bytes written 0	
Rows read 2,750,394,456	Rows written 0	
Files read 1,837	Files written 0	

[See query profile >](#)

 See longest operations for this query

Query wall-clock duration ⓘ

Total wall-clock duration 26 s 78 ms

> Scheduling ⓘ	18%	4 s 683 ms
Optimizing query & pruning files ⓘ	15%	3 s 961 ms
Executing ⓘ	67%	17 s 434 ms

Start time Jan 02, 2026, 06:44:39 PM GMT+02:00

End time Jan 02, 2026, 06:45:05 PM GMT+02:00

Result fetching by client ⓘ 98 ms

[See query profile](#)

În Spark datele pot fi: citite o data, păstrate în memorie, refolosite ulterior

Acesta este **un avantaj major față de Hadoop MapReduce**, unde la fiecare job se recitea datele de pe disc fără reutilizare. **În Databricks, acest mecanism este automat pentru SQL Warehouse.**

Finished

mironela.pirnaeu@prof.utm.ro · UTM Shared SQL Warehouse - 2XS · Run on DBSQL

ID: 01f0e7fa-546c-1b9f-8cf0-490a10a0761e



este **identificatorul unic al interogării SQL** care a fost rulată în Databricks. Acest ID este generat automat pentru **fiecare execuție** și are rolul de a urmări, monitoriza și diagnostica execuția unui job Spark asociat acelei interogări.

În Databricks interogarea SQL → declanșează un **job Spark**, ID-ul este „cheia” care leagă textul interogării, jobul Spark, stage-urile și task-urile.

ID-ul spune - acest job Spark a fost pornit de această interogare.

```
SELECT  
  ss_store_sk,  
  SUM(ss_quantity) AS total_quantity  
FROM samples.tpcds_sf1000.store_sales  
GROUP BY ss_store_sk;
```

Această interogare declanșează un **job Spark distribuit** care implică **scanarea completă a tabelului store_sales**, deoarece coloana ss_quantity trebuie citită pentru fiecare rând. Tabelul este împărțit în multiple **partiții**, fiecare fiind procesată de un task Spark. La nivel conceptual, execuția urmează modelul **MapReduce** vezi cursul 3:

- **Map:** fiecare task citește rândurile din partiția sa și calculează sume parțiale pentru fiecare ss_store_sk
- **Shuffle:** rezultatele intermediare sunt redistribuite între noduri pe baza cheii ss_store_sk
- **Reduce:** pentru fiecare magazin, sumele parțiale sunt agregate într-un rezultat final

Această etapă de **shuffle** este specifică operațiilor GROUP BY și reprezintă una dintre cele mai costisitoare operații în Big Data, deoarece implică transfer de date între nodurile clusterului.

- HDFS (conceptual): datele sunt distribuite în mai multe fișiere
- MapReduce: map → shuffle → reduce
- Spark: execută aceleași etape, dar **in-memory**, fără scrieri repetate pe disc
- Procesare: **batch**, pe întregul set de date

Această interogare realizează o agregare Big Data folosind GROUP BY, declanșând un job Spark distribuit cu etapă de shuffle. Fiecare partiție este procesată în paralel, iar rezultatele sunt agregate conform modelului MapReduce.

Add visualization

	1^2_3 ss_store_sk	1^2_3 total_quantity
1	961	270277665
2	67	270285186
3	619	270645932
4	536	270471239
5	452	270634557
6	439	270849123

↓

▼

502 rows | 0.08s runtime



Databricks poate crea vizualizări **doar dintr-un rezultat SQL**, nu dintr-un tabel brut, nu din date neagregate. Graficul apare după ce rulăm interogarea SQL și folosim opțiunea **Add visualization** din zona de rezultate, unde Databricks permite vizualizarea rezultatului sub formă de chart. Graficul de tip scatter reprezintă rezultatul unei agregări Big Data, unde fiecare punct corespunde unui magazin, iar axa verticală indică suma cantităților vândute. Datele afișate sunt obținute după procesarea distribuită a miliardelor de înregistrări din tabelul store_sales. **Un punct = un magazine**. Pentru fiecare punct:

- poziția pe axa X → magazinul (ss_store_sk)
- poziția pe axa Y → suma cantităților vândute (total_quantity)

```
SELECT  
SUM(ss_quantity) AS total_quantity  
FROM samples.tpcds_sf1000.store_sales  
WHERE ss_quantity > 10;
```

Această interogare pornește tot un job Spark distribuit, însă față de exemplul anterior introduce o etapă suplimentară de filtrare. Spark citește partițiile tabelului și aplică condiția WHERE `ss_quantity > 10` la nivel de task.

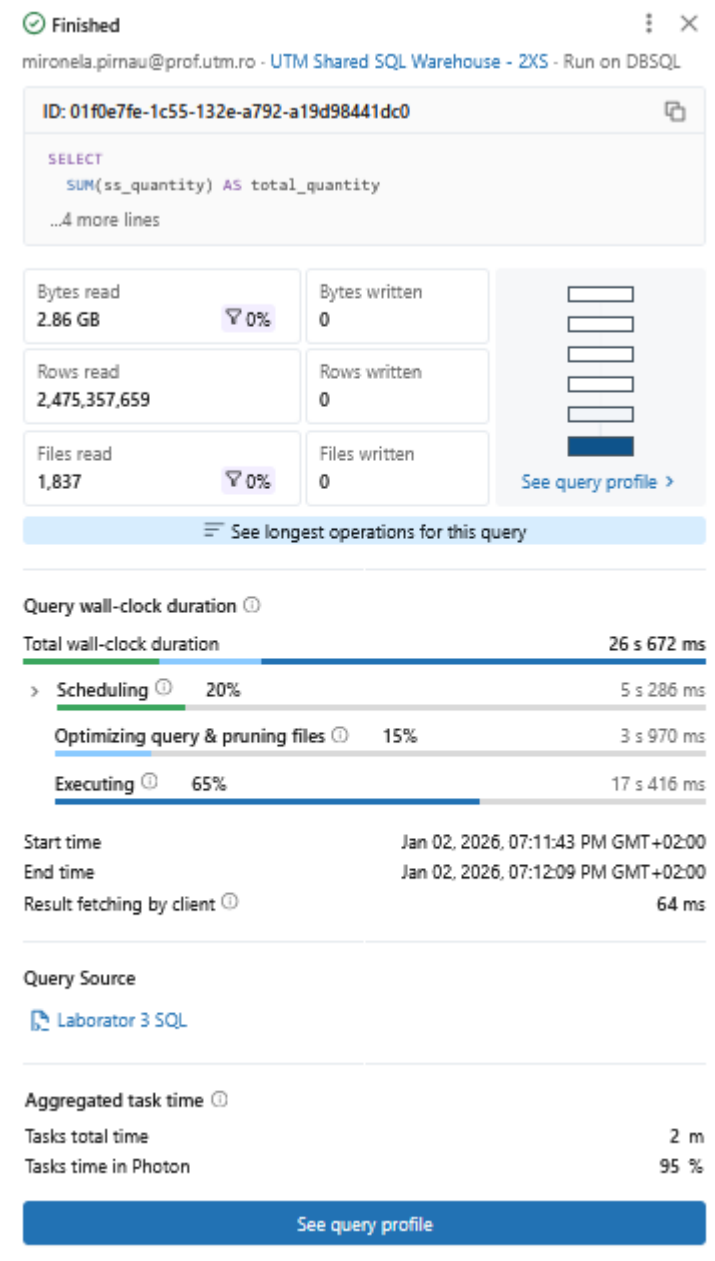
Datele sunt citite distribuit, filtrarea este realizată local, pe fiecare partiție, doar rândurile care respectă condiția participă la agregare. Deși interogarea tot implică un full scan logic, volumul de date procesat ulterior este mai mic, ceea ce duce la o execuție mai eficientă.

În Hadoop MapReduce: fiecare filtru ar fi implicat scrieri intermediare pe disc

În Spark filtrarea și agregarea sunt combinate într-un singur plan de execuție, datele sunt păstrate în memorie cât timp este posibil.

➔ avantajul Spark față de MapReduce clasic, așa cum este explicat în cursul 3.

Prin utilizarea filtrării înainte de agregare, Spark reduce volumul de date procesat, păstrând execuția distribuită, dar îmbunătățind performanța față de un full scan nefiltrat.



```
SELECT COUNT(ss_quantity)
FROM samples.tpcds_sf1000.store_sales;
```

Această interogare realizează numărarea valorilor nenule din coloana `ss_quantity`. Spre deosebire de interogarea `COUNT(*)`, care poate fi optimizată prin utilizarea metadatelor tabelului, operația `COUNT(ss_quantity)` necesită citirea efectivă a coloanei respective din fiecare rând al tabelului. În acest caz, Spark este obligat să parcurgă toate partițiile tabelului `store_sales`, declanșând un job Spark distribuit de tip batch. Fiecare task Spark citește datele din partiția sa și numără valorile locale, iar rezultatele parțiale sunt ulterior agregate într-un

rezultat final. Nu toate operațiile de numărare sunt echivalente din punct de vedere al performanței și că tipul funcției utilizate influențează modul de execuție al jobului Big Data.

În acest caz nu apar fișierele citite deoarece interogarea este servită din cache sau din rezultate reutilizate. Deși COUNT pe coloană implică teoretic citirea datelor, în SQL Warehouse serverless Databricks optimizează execuția și evită recitirea fișierelor dacă informația este deja disponibilă.

Procesarea Big Data poate genera costuri ridicate în producție, însă în Databricks SQL Warehouse serverless resursele sunt limitate și optimizate automat. Cache-ul, oprirea automată și dimensiunea redusă a warehouse-ului previn costuri necontrolate în contextul laboratorului.

```
SELECT  
  ss_store_sk,  
  ss_item_sk,  
  SUM(ss_quantity) AS total_quantity  
FROM samples.tpcds_sf1000.store_sales  
GROUP BY ss_store_sk, ss_item_sk;
```

Această interogare extinde agregarea Big Data prin utilizarea unui GROUP BY compus, bazat pe două chei: magazin (ss_store_sk) și produs (ss_item_sk). Execuția implică scanarea completă a tabelului store_sales, deoarece coloana ss_quantity trebuie citită pentru fiecare rând.

La nivel conceptual, interogarea urmează modelul MapReduce prezentat vezi cursul 3. În faza de tip *Map*, fiecare task Spark procesează o partiție și calculează sume parțiale pentru fiecare combinație magazin–produs. În etapa de *Shuffle*, datele sunt redistribuite între noduri pe baza cheilor de grupare, iar în faza *Reduce* sunt agregate rezultatele finale.

Acest exemplu este important deoarece evidențiază impactul operațiilor GROUP BY asupra volumului de date transferat între noduri și asupra costului computațional al procesării Big Data.

```
SELECT  
  ss_store_sk,  
  SUM(ss_quantity) AS total_quantity  
FROM samples.tpcds_sf1000.store_sales  
GROUP BY ss_store_sk  
ORDER BY total_quantity DESC
```

LIMIT 10;

Această interogare selectează primele 10 magazine cu cele mai mari cantități vândute. Deși este utilizată clauza LIMIT, Spark este obligat să proceseze întregul set de date pentru a calcula agregările necesare înainte de sortare.

Agregarea și sortarea sunt realizate asupra tuturor partițiilor tabelului, iar limitarea se aplică **doar asupra rezultatului final**, nu asupra datelor brute. Acest exemplu este relevant din punct de vedere didactic deoarece demonstrează faptul că LIMIT nu reduce volumul de date procesat într-un context Big Data, ci doar cantitatea de date returnată utilizatorului.

Analiza planului de execuție cu EXPLAIN

Prin utilizarea comenzii EXPLAIN, se poate analiza modul în care Spark construiește planul de execuție al unei interogări Big Data. Spark generează mai întâi un plan logic, care descrie transformările necesare asupra datelor, și ulterior un plan fizic, care stabilește modul concret de execuție distribuită.

Acest exemplu permite observarea etapelor de scanare, agregare și optimizare realizate de motorul Spark și evidențiază rolul optimizatorului Catalyst în planificarea joburilor Big Data. Analiza planului de execuție ajută la înțelegerea modului în care o interogare SQL aparent simplă este transformată într-un job distribuit complex.

EXPLAIN

SELECT SUM(ss_quantity)

FROM samples.tpcds_sf1000.store_sales;

```

Add parameter
Table +
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- == Initial Plan ==
  ColumnarToRow
  +- PhotonResultStage
    +- PhotonAgg(keys=[], functions=[finalmerge_sum(merge sum#133561) AS sum(ss_quantity)#133431])
      +- PhotonShuffleExchangeSource
        +- PhotonShuffleMapStage ENSURE_REQUIREMENTS, [id=&#6999]
          +- PhotonShuffleExchangeSink SinglePartition
            +- PhotonAgg(keys=[], functions=[partial_sum(ss_quantity#13330) AS sum#133561])
              +- PhotonProject [ss_quantity#13330]
                +- PhotonScan parquet samples.tpcds_sf1000.store_sales[ss_quantity#13330,ss_sold_date_sk#13320] DataFilters: [], DictionaryFilters: [], Format: parquet, Location: PreparedDeltaFileIndex(1 paths)[s3://system-t
ables-prod-eu-central-1-uc-metastore-bucket/metastor..., OptionalDataFilters: [], PartitionFilters: [], ReadSchema: struct<ss_quantity:int>, RequiredDataFilters: []

== Photon Explanation ==
The query is fully supported by Photon.
== Optimizer Statistics (table names per statistics state) ==
missing =
partial =
full    = store_sales
```

În cadrul procesării MapReduce, după etapa de mapare, datele sunt redistribuite între nodurile de lucru în funcție de cheia de grupare, astfel încât agregarea finală să poată fi realizată corect.

În Spark, această redistribuire a datelor între noduri este denumită shuffle, însă din punct de vedere conceptual ea corespunde fazei de reorganizare a datelor necesară agregării, descrisă în cursul 3 în cadrul modelului MapReduce

În Databricks, termenul „shuffle” se referă la procesul **costisitor de redistribuire a datelor între nodurile de lucru (executors) și partiții**, necesar pentru anumite tipuri de operații distribuite. Acest proces este esențial pentru procesarea paralelă corectă, dar are un impact semnificativ asupra performanței, deoarece implică **transfer de date prin rețea, scrieri și citiri pe disc și consum de memorie**.

Din punct de vedere conceptual, shuffle-ul apare atunci când Spark trebuie să aducă **date înrudite (cu aceeași cheie)** pe același nod pentru a putea efectua o operație globală, cum ar fi agregarea sau îmbinarea datelor.

Shuffle-ul (numit uneori și *exchange*) reprezintă momentul în care Spark **mută datele între noduri**. Acest lucru este necesar pentru operații precum:

- GROUP BY
- JOIN
- DISTINCT
- AGGREGATE

În aceste cazuri, datele nu mai pot fi procesate independent în fiecare partiție, ci trebuie **reorganizate** astfel încât toate valorile cu aceeași cheie să ajungă pe același executor.

Transformări înguste (narrow transformations) Exemple: map, filter → fiecare partiție este procesată independent → **NU apare shuffle**

- **Transformări largi (wide transformations)**

Exemple: GROUP BY, JOIN, DISTINCT → datele trebuie redistribuite între noduri → **APARE shuffle**

Shuffle-ul este specific transformărilor „largi” și este considerat una dintre cele mai costisitoare operații în Spark.

De ce este shuffle-ul costisitor

Shuffle-ul implică simultan mai multe tipuri de operații scumpe:

- **I/O de rețea** – datele sunt transferate între noduri
- **I/O pe disc** – datele sunt scrise și citite din fișiere temporare (shuffle write / shuffle read)
- **Serializare / deserializare** – datele trebuie convertite pentru transfer
- **Consum de memorie** – buffer-e pentru datele intermediare

Din acest motiv, shuffle-ul este adesea **principala sursă de probleme de performanță** în aplicațiile Big Data.

Cum funcționează shuffle-ul – exemplu simplu (DISTINCT)

Etapa 1 – Procesare locală

Fiecare executor:

- procesează partiția sa
- determină valorile distincte locale

Această etapă se face **fără comunicare între noduri**.

Etapa 2 – Redistribuire globală

Pentru a obține un rezultat global corect:

- datele sunt **redistribuite între executors**
- fiecare valoare ajunge la nodul responsabil pentru cheia sa
- datele sunt scrise pe disc și citite din nou

Abia după această etapă se poate obține rezultatul final DISTINCT la nivel global.

Optimizări oferite de Databricks pentru shuffle

Databricks include mai multe mecanisme care reduc impactul shuffle-ului:

Low Shuffle Merge

- utilizat în special pentru operații MERGE pe Delta Lake
- separă datele modificate de cele nemodificate
- reduce semnificativ volumul de date redistribuite
- îmbunătățește viteza de execuție

Volume dedicate pentru shuffle

- se pot folosi discuri SSD dedicate pentru datele de shuffle
- previne supraîncărcarea discurilor standard
- îmbunătățește stabilitatea și performanța

Shuffle-ul este mecanismul prin care Spark redistribuie datele între noduri pentru a permite procesarea corectă a operațiilor globale, precum agregările și îmbinările. Deși este esențial pentru paralelism, shuffle-ul reprezintă una dintre cele mai costisitoare etape ale procesării Big Data, motiv pentru care Databricks oferă optimizări dedicate pentru reducerea impactului său asupra performanței.

```
SELECT
  ss_store_sk,
  SUM(ss_quantity) AS total_quantity
FROM samples.tpcds_sf1000.store_sales
GROUP BY ss_store_sk;
```

În cadrul acestei interogări, datele sunt procesate inițial în mod distribuit, fiecare executor lucrând pe partiția sa. După etapa de mapare, în care se calculează sume parțiale pentru fiecare magazin, datele trebuie redistribuite între nodurile de lucru astfel încât toate valorile asociate aceleiași chei (ss_store_sk) să ajungă pe același nod.

Această redistribuire a datelor corespunde, din punct de vedere conceptual, fazei descrise în cursul 3 din modelul MapReduce, în care rezultatele parțiale sunt reorganizate pentru a permite agregarea finală. În Spark și Databricks, acest proces este denumit *shuffle*.

```
SELECT *
FROM samples.tpcds_sf1000.store_sales
WHERE ss_quantity > 10;
```

Această interogare aplică o operație de filtrare asupra datelor. Fiecare partiție este procesată independent, iar condiția WHERE este evaluată local, fără a fi necesară comunicarea între nodurile de lucru.

Conform clasificării transformărilor Spark, această operație este o **transformare îngustă (narrow transformation)**. În acest caz nu apare redistribuirea datelor între noduri și, implicit, nu are loc un shuffle.

```
SELECT DISTINCT ss_store_sk
FROM samples.tpcds_sf1000.store_sales;
```

Execuția acestei interogări are loc în două etape. În prima etapă, fiecare executor determină valorile distincte locale din partiția sa, fără comunicare cu alte noduri. Această etapă este similară fazei de mapare din modelul MapReduce.

În a doua etapă, pentru a obține un rezultat global corect, datele sunt redistribuite între executors, astfel încât fiecare valoare distinctă să poată fi identificată la nivelul întregului set de date. Această etapă implică scrierea și citirea datelor intermediare pe disc și transferul acestora prin rețea, reprezentând procesul de shuffle.

```
SELECT
  ss.ss_store_sk,
```

```

d.d_store_name,
SUM(ss.ss_quantity) AS total_quantity
FROM samples.tpcds_sf1000.store_sales ss
JOIN samples.tpcds_sf1000.store_d
ON ss.ss_store_sk = d.d_store_sk
GROUP BY ss.ss_store_sk, d.d_store_name;

```

În cazul operației JOIN, datele din cele două tabele trebuie aduse împreună pe baza cheii de legătură. Deoarece datele sunt distribuite pe mai multe noduri, Spark este obligat să redistribuie rândurile astfel încât cele cu aceeași cheie să ajungă pe același executor.

Această redistribuire este un exemplu tipic de transformare largă (wide transformation) și implică un shuffle. Costul acestei operații este ridicat, deoarece presupune transfer de date prin rețea și utilizarea intensivă a resurselor de calcul și stocare.

```

SELECT
ss_store_sk,
COUNT(*) AS total_sales
FROM samples.tpcds_sf1000.store_sales
GROUP BY ss_store_sk;

```

Această interogare declanșează un job Spark distribuit care necesită redistribuirea datelor în funcție de cheia de grupare. În timpul execuției, Spark efectuează operații costisitoare de I/O de rețea, I/O pe disc și serializare a datelor.

Acest exemplu evidențiază de ce shuffle-ul este considerat una dintre cele mai costisitoare etape ale procesării Big Data și de ce este principalul factor care influențează performanța aplicațiilor distribuite.

```

SELECT
ss_store_sk,
SUM(ss_quantity) AS total_quantity
FROM samples.tpcds_sf1000.store_sales
GROUP BY ss_store_sk;

```

Pentru astfel de interogări care implică redistribuirea datelor, Databricks oferă mecanisme de optimizare, precum Low Shuffle Merge sau utilizarea unor volume dedicate pentru datele de shuffle. Aceste optimizări nu schimbă logica procesării, ci reduc volumul de date redistribuite și impactul operațiilor de I/O asupra performanței.

Prin aceste exemple se demonstrează modul în care redistribuirea datelor este necesară pentru realizarea operațiilor globale în procesarea Big Data. Deși termenul „shuffle” este specific Spark și Databricks, conceptul corespunde fazei de reorganizare a datelor din modelul MapReduce prezentat în cursul 3. Exemplele evidențiază atât situațiile în care shuffle-ul apare, cât și impactul său asupra performanței.